



SC4021 Group Assignment: Opinion Search Engine on Smartphones

Group 37

Team Members	Matriculation Number
Kenneth Lemuel	U2222928H
Kim Minhwan	U2022178H
Javier Tan Choon Heng	U2421852D
Ivan Soji Varghese	U2220650L
Kalidoss Madhumitha	U2222332A
Junnoske Kita	U2422700E

Table of Contents

Crawling.....	3
1. Introduction to Crawling.....	3
1.1 Methods of Crawling and Storage of Corpus.....	3
1.2 What can be Retrieved From the Corpus.....	4
1.3 Corpus Size, Records, Words and Types.....	5
Indexing.....	7
2. Introduction to Indexing.....	7
2.1 Design of UI.....	7
2.2 Write five queries' results and speed measurement.....	7
3. Innovations for enhancing the indexing and ranking.....	9
1. Multifaceted Search.....	9
2. Timeline and date-range constrained search.....	10
3. Enhanced search analytics (pie chart, category chart, word cloud, trend line).....	11
4. Comparative sentiment analysis between two queries.....	12
5. Robust model-aware query normalization and disambiguation.....	13
6. Out-of-scope model detection to prevent misleading retrieval.....	14
7. Fixed hybrid re-ranking for opinion quality.....	15
8. Integration of final sentiment labels from the sentiment classification pipeline.....	16
Classification.....	17
4. Introduction to Classification.....	17
4.1 Motivation for Classification Approach.....	17
Overall Pipeline Design.....	17
Subjectivity Detection.....	17
Sarcasm Detection.....	18
Aspect Detection.....	18
Hybrid Sentiment Classification Approach.....	19
General Sentiment Analysis.....	19
Aspect-based Sentiment Analysis (ABSA).....	20
4.2 Data Preprocessing: Design & Justification.....	20
4.3 Evaluation Dataset.....	22
4.4. Evaluation Metrics.....	22
4.5 Random Accuracy Test.....	23
4.6 Performance Metrics and Scalability.....	25
5. Innovations in the Classification Pipeline.....	26
Fine-grained Classification (Aspect-based Sentiment Analysis).....	26
Hybrid Classification (Symbolic and Subsymbolic AI in Subjectivity Detection).....	26
Enhanced Classification (Sarcasm Detection).....	27
Ablation Studies.....	27
Links for Submissions.....	29
Youtube Video.....	29
Dataset.....	29
Source Codes and Libraries.....	29
Appendix A: Screenshots of UI.....	30
References.....	33

Crawling

1. Introduction to Crawling

The purpose of the data crawling component was to build a structured YouTube comment corpus that's centered on a fixed set of smartphone devices such that later project stages can perform sentiment classification, indexing, querying and evaluation on a consistent and reproducible dataset. In the main branch, this workflow is implemented mainly in *crawler/youtube_pipeline.py* and *crawler/prepare_annotation_dataset.py*, with the staged outputs written to the data directory.

1.1 Methods of Crawling and Storage of Corpus

The corpus was collected from YouTube as the platform contains large volumes of naturally occurring discussions about newly released consumer devices. Compared with curated benchmark datasets, YouTube comments better capture noisy, informal, opinion-rich user discourses which include comparisons, complaints, recommendations and reactions to specific phone features. This makes it suitable for the projects' objective of building a searchable corpus of user opinions.

The crawler was set up to use YouTube Data API v3 as the main collection interface. In the code, HTTP requests are handled through the Python requests library, API credentials are then loaded using python-dotenv, file paths are managed with pathlib and the dataset transformation was handled later with pandas. Regular expressions are used extensively to enforce bucket scope, match device titles and score phone relevancy. These choices made the crawler more lightweight, scriptable and easier to run.

The crawl scope was intentionally restricted to eight device buckets that were “on the same level” or often compared to each other:

- iPhone 17
- iPhone 17 Pro
- Galaxy S26
- Galaxy S26 Ultra
- Pixel 10
- Pixel 10 Pro
- Xiaomi 15
- Xiaomi 15 Ultra

Each bucket is defined in BucketRule objects inside of *crawler/youtube_pipeline.py* together with family labels, variant labels, include patterns and exclude patterns. This prevented the more obvious cross-contamination between closely related models such as the base versus Pro or base versus Ultra variants.

Seed videos were discovered using fixed query templates. For each device the crawler searched four query categories: ‘review’, ‘camera’, ‘battery’ and ‘issue’, which was implemented as ‘<term> review’, ‘<term> camera’, ‘<term> battery’, and ‘<term> overheating issue problem bug’. The logic also constrained results by publication date and minimum video duration, and it capped how many videos

could be kept per query and per channel. These constraints were included to reduce duplicate coverage and avoid over-reliance on a few channels or very short videos with poor discussion quality.

Once the seed videos were selected, the crawler collected YouTube comments and replies into *data/comments_raw.csv*. This raw file preserves source metadata such as video_id, bucket, family, variant and search_query, alongside comment-level fields such as comment_id, reply information, timestamps and like counts. The raw corpus was then cleaned into *data/comments_clean.csv*, after which the *crawler/prepare_annotation_dataset.py* applied language filtering, relevance scoring and rule-based comment category assignment to generate *data/comments_scored.csv*, *data/comments_relevant.csv*, *data/relevance_summary.csv* and *data/annotation_candidates.csv*.

The staged storage strategy is important as it makes the entire crawl auditable. It is listed as such:

- *seed_videos.csv* records the selected videos
- *comments_raw.csv* keeps the untouched crawl output
- *comments_clean.csv* stores the normalized comments
- *comments_scored.csv* keeps engineered signal such as is_english, relevance_score and comment_category
- *comments_relevant.csv* is the final sentiment ready corpus, *relevance_summary.csv* provides per-bucket diagnostics
- *annotation-candidates.csv* provides a subset for the team to manually annotate and evaluate our final model on.

1.2 What can be Retrieved From the Corpus

The crawled corpus is designed to support retrieval of opinion-oriented information rather than generic document search. Because each retained comment stores device metadata and a rule-based comment-category, our team can search for comments by phone model, family and aspect. This makes it such that the corpus is suitable for applications such as public-opinion exploration, device comparison, feature-specific complaint discovery and sentiment-aware search.

Examples of information that users may want to retrieve includes:

- overall opinions about a phone
- battery complaints
- camera praise
- price sensitivity
- software-related issues
- cross-device comparisons

These are just examples of some things that can be retrieved. This is useful for several downstream scenarios such as a potential buyer may want to inspect complaints before buying a phone or when a reviewer may want to analyze which aspects attract the most negative reactions. Such that the search system can retrieve comments matching a specific device-aspect combination rather than returning the entire discussion space.

Representative retrieval applications and sample queries are as follows:

- Overall sentiment exploration: “Galaxy S26 Ultra Overall”
- Aspect-focused retrieval: “iPhone 17 battery”
- Complaint analysis: “Pixel 10 Pro” and then search for “Negative” sentiment
- Comparison support: Model A “Xiaomi 15 Ultra” Model B “Pixel 10 Pro”
- Price sensitivity search: iPhone 17 Pro price”

These are sample queries that are supported by the way crawler stores data and how the indexing/querying part is done. Each comment is linked to a device bucket and later enriched with a comment-level aspect category such that the search engine does not have to rely on raw keyword matching alone. The crawling stage therefore directly enables the indexing and querying part expected in the later system.

1.3 Corpus Size, Records, Words and Types

The current main-branch pipeline produces several staged datasets, each representing a different level of filtering. The corpus statistics in the table 1 below are based on the generated files currently stored in the repository. It is important to distinguish between the fully cleaned corpus and final sentiment-ready subset as the latter is the corpus actually used for later classification and searching. This is all in the data folder.

Table 1: Summarizes main dataset artifacts and record counts.

File	Records	Meaning
seed_videos.csv	173	Selected seed videos after scoped YouTube search
comments_raw.csv	26,186	Raw collected comments and replies
comments_clean.csv	25,867	Clean comment corpus after normalization
comments_scored.csv	25,867	Clean comments with relevance features
comments_relevant.csv	7,796	Final sentiment-ready corpus
annotation_candidates.csv	1,000	Evaluation subset prepared for manual annotation

The raw crawl contains 26,186 comment records and after cleaning the corpus contains 25,867 records, this is shown in table 2. After proper English language filtering and further relevance scoring, the final sentiment-ready corpus contains 7,796 records. This means the crawler deliberately sacrifices volume in exchange for a more useful dataset.

Table 2: Reports the world-level statistics required by assignment

Corpus view	Records	Word tokens	Unique word types	Average words per record
Cleaned corpus (comments_clean.csv)	25,867	565,024	24,066	21.84
Sentiment-ready corpus (comments_relevant.csv)	7,796	277,047	13,561	35.54

For clarification, the word and type counts above were computed by lowercasing the comment text and tokenizing it with a simple alphanumeric regex tokenizer. Under this definition, the cleaned corpus contains 565,024 word tokens and 24,066 unique word types, while the final sentiment-ready corpus contains 277,047 word tokens and 13,561 unique word types. Reporting both views is useful as it shows not only the size of the original collected material, but also the size of the practically usable corpus after filtering.

Indexing

2. Introduction to Indexing

The indexing component builds a searchable inverted index over the crawled corpus using Woosh, enabling full-text retrieval with BM25 scoring, stemming, and structured filtering. A Streamlit-based UI provides faceted search and comparative analysis, allowing users to query and interpret opinions efficiently.

2.1 Design of UI

We went with Woosh over heavier options like Solr or Elasticsearch, mainly because we want to keep it simple, it runs in-process alongside the app, no extra services to manage. Solr needs a Java runtime and servlet container which is an overkill for 10k Youtube comments.

Our schema uses a StemmingAnalyzer on all free-text fields so variants like “charging” and “charge” get collapsed at index time. Queries go through MultifieldParser with BM25 scoring across fields like comment text, phone model, brand, and video title. For faceted filtering (category, sentiment, date range, reply status), we build structured queries using Woosh’s Term, DateRange, and And objects directly, so filtering happens inside the index rather than retrieval.

The downside to Woosh is that we will not be able to scale to millions of records or handle concurrent users well. But it makes sense to use Woosh for our use case; single-user evaluation on a moderately sized data set, as mentioned in our assignment brief.

The assignment brief also said to keep the UI simple and that a sophisticated interface isn’t expected. Streamlit library fits perfectly to the requirement well since it lets us build interactive web apps entirely in Python without touching frontend frameworks or JavaScript toolchains.

Our interface includes a search bar, sidebar facets for brand/model/category/date filtering and three tabs: a summary view with Plotly charts and word clouds, a ranked results view with paginated comment cards, and sentiment comparison tool that runs two queries side-by-side. Everything is driven by pandas DataFrame passed straight from Woosh into Streamlit’s layout and charting components, so there’s no serialization layer or API boundary sitting between the backend and the UI. Compared to Django or Flask, this lets us iterate faster while keeping the entire codebase in one Python package.

[Refer to Appendix A for the screenshots of the full UI](#)

2.2 Write five queries’ results and speed measurement

The 5 queries made show a good range of possible query-result pairs, where model, brand and topic can be queried and results can be narrowed or widened. It also shows when a model that is outside the

scope of models that are indexed is queried, The queries were conducted 5 times, and the average latency was calculated. All query latencies were found to be at most 300-400ms, which is an acceptable speed without a significantly noticeable lag.

Table 3: Query Results Summary

Query Text	Avg Latency (ms)	Result Count	Comments
“iPhone 17 Pro”	309.08	539	Model detected - bucket fixed to iPhone 17 Pro Brand detected - family fixed to Apple
“Samsung Galaxy S26 Ultra battery”	155.25	232	Model detected - bucket fixed to Galaxy S26 Ultra Brand detected - family fixed to Samsung Topic detected - category fixed to battery
“Xiaomi 15 camera”	155.35	91	Model detected - bucket fixed to Xiaomi 15 Brand detected - family fixed to Xiaomi Topic detected - category fixed to camera
“Google”	101.81	1675	Brand detected - family fixed to Google
“Apple iPhone 15”	1.54	0	Brand detected - family fixed to Apple Out-of-scope model detected - iphone 15

3. Innovations for enhancing the indexing and ranking

Our final system introduces several improvements over the first version of the search engine. These innovations were designed to solve concrete retrieval problems such as noisy matches, weak interpretability, and limited analytical depth.

These innovations will be described in the following format:

- Problem
- Innovation
- Why it matters

1. Multifaceted Search

Problem: A plain keyword often returns broad and mixed results. For example, searching ‘battery’ can retrieve comments from multiple brands and models, making analysis unfocused.

Innovation: Added facet-driven filtering by brand, model, and topic/category, with query-aware inference that can automatically constrain results when model or brand intent is clearly detected (Figure 2 & 3).

Why it matters: This improves precision and lets users quickly zoom into a specific evidence set, such as brand = Apple | model = iPhone 17 | category = battery (Figure 1).

Figure 1: Example of results for multifaceted queries

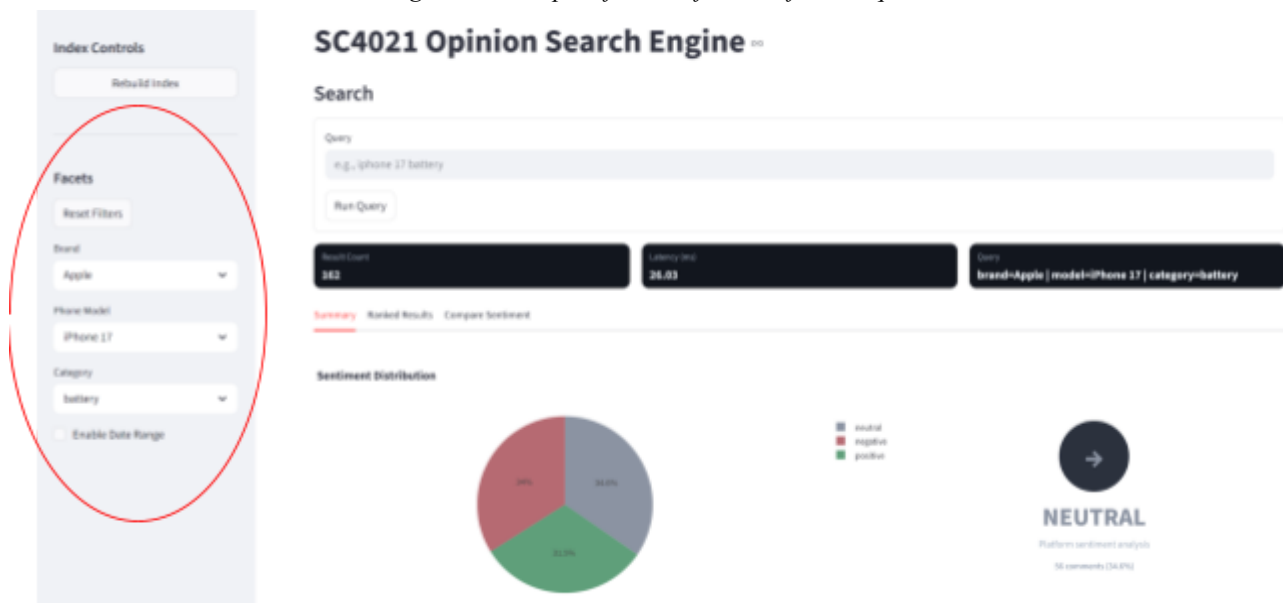


Figure 2: Snippet from indexing.py, search()

```
query_parts = [base_query]
if category != "All":
    query_parts.append(Term("comment_category", str(category)))
if sentiment != "All":
    query_parts.append(Term("suggested_sentiment_label", str(sentiment)))
if not include_replies:
    query_parts.append(Term("is_reply", False))
```

Figure 3: Snippet from inference.py, the automatic brand/model detection

```
def infer_bucket_from_query(query_text: str, all_buckets: list[str]) -> str | None:
    q = (query_text or "").strip().casefold()
    if not q:
        return None

    ordered = sorted(all_buckets, key=lambda x: len(x), reverse=True)
    for bucket in ordered:
        if _find_bucket_match_span(q, bucket, all_buckets) is not None:
            return bucket
    return None
```

2. Timeline and date-range constrained search

Problem: Sentiment and discussions change over time. Without temporal constraints, users cannot isolate periods such as launch week reactions versus later long-term usage feedback.

Innovation: Added date-range filtering using Woosh's DateRange on the published_at DATETIME field and results-over-time visualization (Figure 5).

Why it matters: Users can perform temporally meaningful analysis, for example limiting retrieval to a specific window to study trend shifts (Figure 4).

Figure 4: Example of results for timeline and date-range constrained search

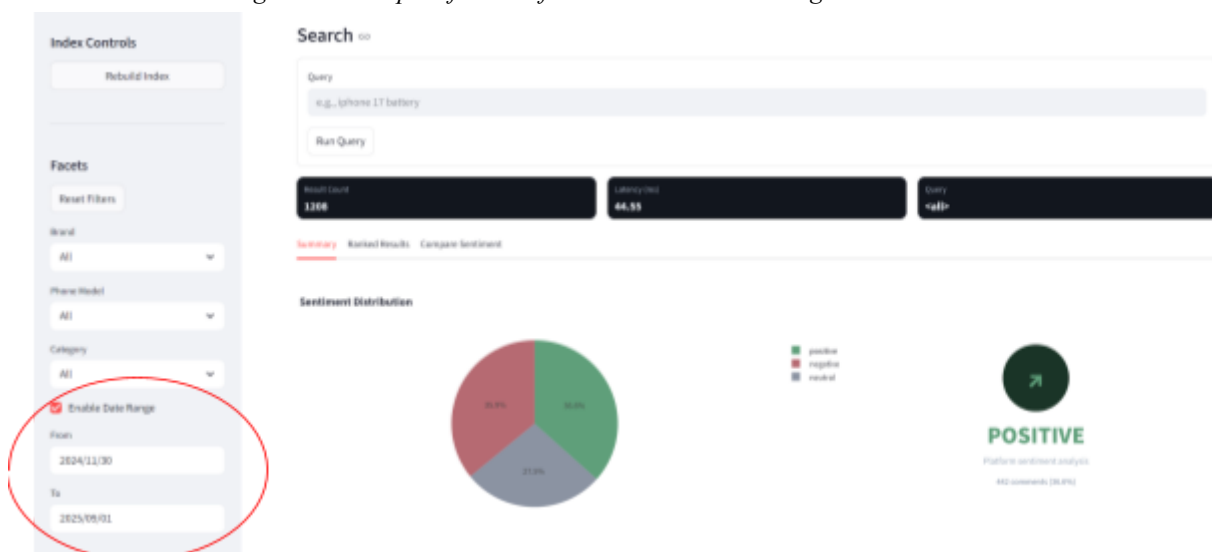


Figure 5: Snippet from indexing.py, search()

```
if enable_date_filter and start_date and end_date:
    if start_date > end_date:
        start_date, end_date = end_date, start_date
    query_parts.append(
        DateRange(
            "published_at",
            datetime.combine(start_date, datetime.min.time()),
            datetime.combine(end_date, datetime.max.time()),
        )
    )
```

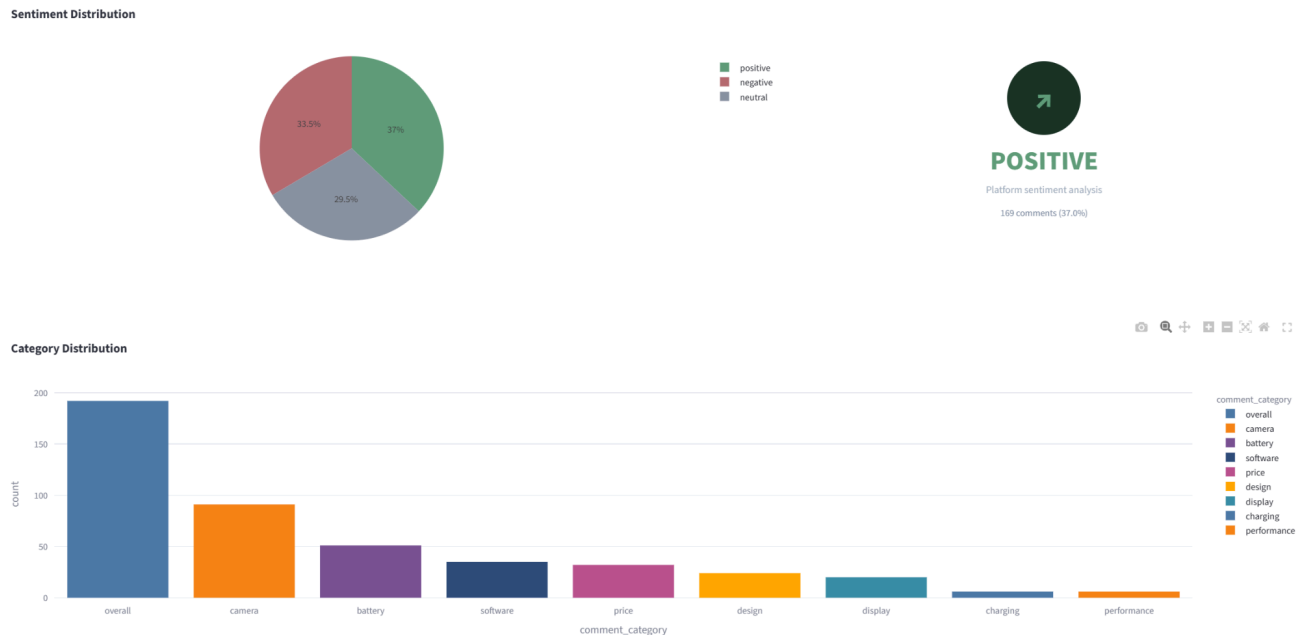
3. Enhanced search analytics (pie chart, category chart, word cloud, trend line)

Problem: Raw ranked comments alone make it difficult to understand high-level sentiment structure and dominant discussion themes.

Innovation: We added summary visual analytics including sentiment distribution, category distribution, temporal trend, and word cloud.

Why it matters: This turns retrieval into insight. Users can move from micro-level evidence (individual comments) to macro-level interpretation (overall mood, dominant topics, topic volume over time) (Figure 6 & 7).

Figure 6: Example of search analytics charts for sentiment distribution and category distribution (micro-level)



Results Over Time

Number of Comments

Date

Jan 2025 Mar 2025 May 2025 Jul 2025 Sep 2025 Nov 2025 Jan 2026 Mar 2026

Why it matters: This supports decision-style analysis such as battery sentiment versus camera sentiment for the same model (Figure 8), or comparing two models on the same feature (Figure 9).

Index Controls

Rebuild Index

Facets

Reset Filters

Brand

All

Phone Model

All

Category

All

☐ Enable Date Range

Compare Two Queries

Compare sentiment distribution for two search queries under the current sidebar filters.

Query A

s26 ultra camera

Query B

s26 ultra battery

Compare Sentiment

Sentiment Comparison

Sentiment	s26 ultra camera	s26 ultra battery
positive	232	116
neutral	232	116
negative	232	116

Query A Results

432

Query B Results

232

Net Sentiment Delta (B - A)

-8.17 pp

metric	s26 ultra camera	s26 ultra battery	Difference (B - A)
Positive %	53.9444	27.5862	-4.3582
Negative %	35.4367	39.2241	3.8075
Net Sentiment %	-3.4722	-11.6379	-8.1657

Figure 9: Example comparing sentiments of Xiaomi 15 and iPhone 17 storage

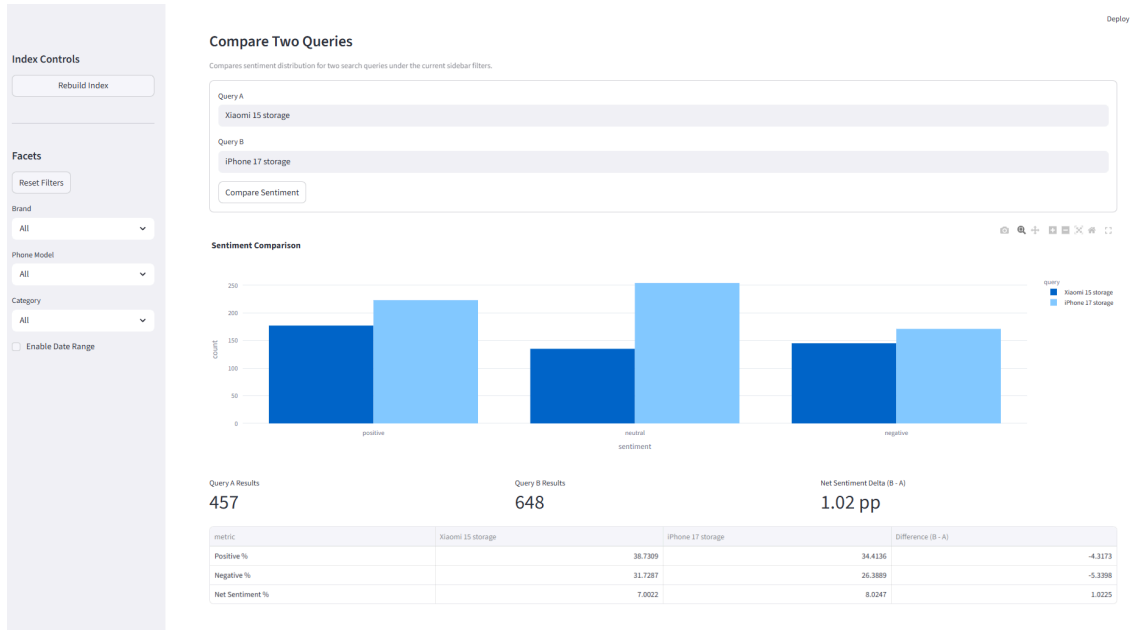


Figure 10: Snippet from compare.py

```
q1_counts = sentiment_counts(q1_filtered)
q2_counts = sentiment_counts(q2_filtered)

query_a_label = (query_a or "").strip() or "Query A"
query_b_label = (query_b or "").strip() or "Query B"
if query_a_label == query_b_label:
    query_b_label = f"{query_b_label} (B)"

labels = ["positive", "neutral", "negative"]
compare_long = pd.DataFrame(
    [{"sentiment": label, "query": query_a_label, "count": q1_counts[label]} for label in labels]
    + [{"sentiment": label, "query": query_b_label, "count": q2_counts[label]} for label in labels]
```

5. Robust model-aware query normalization and disambiguation

Problem: User model queries are noisy and vary in format, for example iphone17pro, iphone 17 pro, iphone-17-pro. Older logic can miss or mix these variants.

Innovation: We introduced strict model matching with flexible tokenization and canonical normalization, plus explicit handling of base versus premium variants (Figure 12).

Why it matters: This reduces false matches and improves ranking quality by aligning query text with canonical indexed model names (Figure 11).

Figure 11: Example comparing query texts “iphone-17-pro” and “iphone17pro”

Search

Query

iphone 17 pro

Run Query

Model detected -> bucket fixed to iPhone 17 Pro
Brand detected -> family fixed to Apple

Result Count

539

Latency (ms)

240.91

Query

iPhone 17 Pro

Search

Query

iphone-17-pro

Run Query

Model detected -> bucket fixed to iPhone 17 Pro
Brand detected -> family fixed to Apple

Result Count

539

Latency (ms)

247.08

Query

iPhone 17 Pro

Figure 12: Snippet from inference.py

```
def normalize_query_to_canonical_model(
    query_text: str,
    bucket: str,
    all_buckets: list[str],
    family: str | None = None,
) -> str:
    query = query_text or ""
    if not query.strip() or not bucket:
        return query_text

    span = _find_bucket_match_span(query, bucket, all_buckets)
    if span is None:
        return query_text

    include_family = False
    if family:
        aliases = _family_aliases(family)
        lowered = query.casefold()
        include_family = not any(re.search(rf"\b{re.escape(alias)}\b", lowered) for alias in aliases)
```

6. Out-of-scope model detection to prevent misleading retrieval

Problem: If a user searches for a model not present in the dataset (for example 'iPhone 16'), a naive system may still return nearest brand-level results, which is misleading.

Innovation: Added out-of-scope detection for unknown models (queries containing a recognized brand plus a model number absent from the index), blocking retrieval and showing an explicit warning instead of silently returning brand-level results (Figure 14).

Why it matters: This improves reliability and transparency by preventing incorrect interpretation of Results (Figure 13).

Figure 13: Example output of out-of-scope model detection

SC4021 Opinion Search Engine

Search

Query

iphone 16

Run Query

Brand detected -> family fixed to Apple
Out-of-scope model detected -> iphone 16

The query references a phone model outside this dataset scope (iphone 16). No results are returned.

Result Count

0

Latency (ms)

3.36

Query

iphone 16

Summary

Ranked Results

Compare Sentiment

No result rows for sentiment summary.

Figure 14: Snippet from inference.py

```
def infer_unknown_model_from_query(
    query_text: str,
    all_buckets: list[str],
    all_families: list[str],
) -> str | None:
    q = (query_text or "").strip()
    if not q:
        return None

    if infer_bucket_from_query(q, all_buckets) is not None:
        return None

    inferred_family = infer_family_from_query(q, all_families)
    if inferred_family is None:
        return None

    lowered = q.casefold()
    has_model_number = bool(re.search(r"\b(?:\d{1,3}|[a-z]\d{1,3})\b", lowered))
    if not has_model_number:
        return None
```

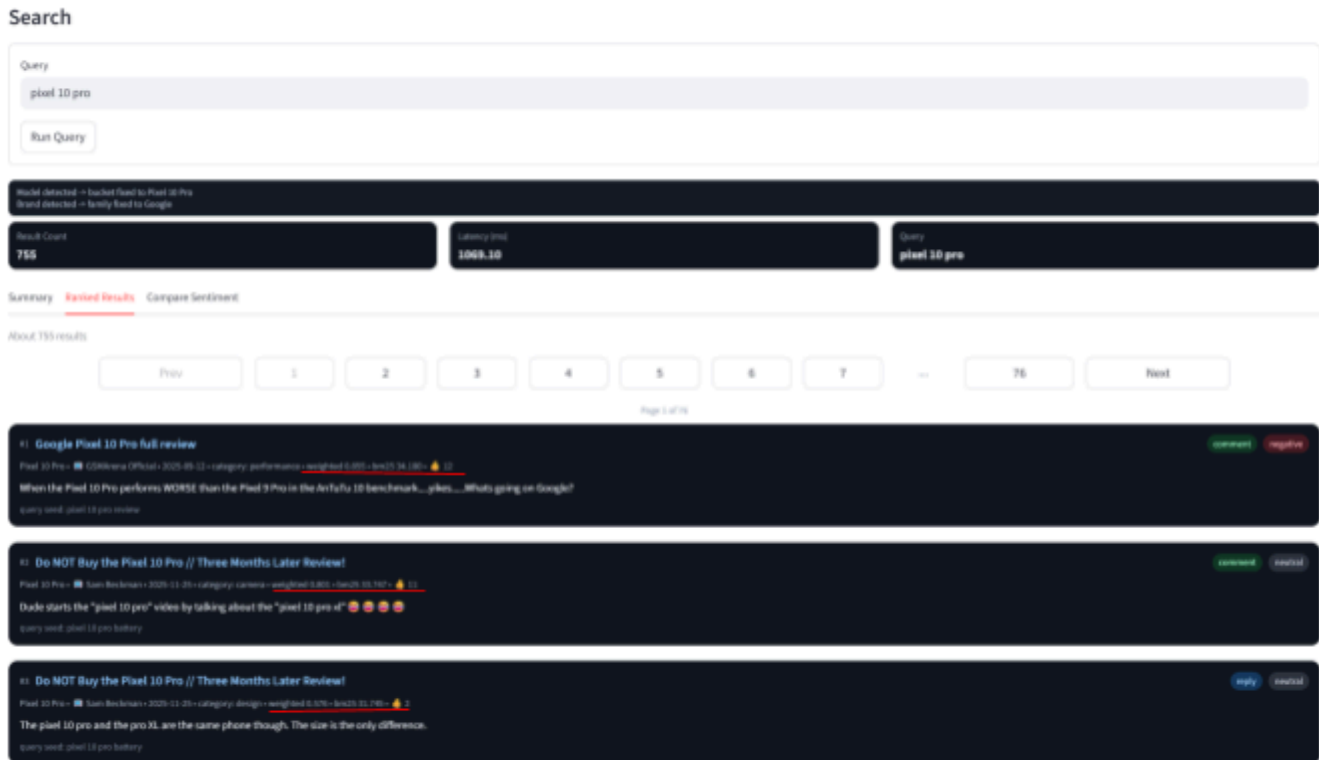
7. Fixed hybrid re-ranking for opinion quality

Problem: Pure lexical ranking can surface comments that are textually similar but less useful as opinion evidence.

Innovation: We use a fixed weighted reranking strategy combining text relevance, relevance score, engagement signal, and quality prior (for example top-level comment preference) (Figure 15).

Why it matters: This produces more informative top-ranked comments and improves result usefulness for sentiment interpretation.

Figure 15: Example of weighted ranking results



8. Integration of final sentiment labels from the sentiment classification pipeline

Problem: Early versions relied on placeholder sentiment labels, reducing trust in summary and comparison outputs.

Innovation: We joined final sentiment outputs by comment ID from the final sentiment classification output file into the main retrieval pipeline.

Why it matters: All sentiment visualizations and comparisons now reflect the true final classifier output, improving validity of findings.

Classification

4. Introduction to Classification

The classification pipeline of this project focuses on identifying and analysing opinions from the crawled comments, including both general sentiments about smartphones and sentiments related to specific features such as camera and battery to enhance the functionality of the search engine. This allows users to not only retrieve comments relevant to their queries, but also to gain meaningful insights into the overall sentiment distribution of opinions related to their queries.

4.1 Motivation for Classification Approach

Overall Pipeline Design

Smartphone comments can vary in nature as some are factual, some express overall opinions and others address specific features of smartphones. Because of this diversity, a single sentiment classifier is insufficient, so the pipeline adopts a hybrid, multi-staged sentiment detection approach that routes different types of comments to the most suitable classification approach. The pipeline first separates objective and subjective content, then routes subjective comments (opinions) into either general sentiment analysis or aspect-based sentiment analysis (ABSA) depending on whether the opinion refers to a specific smartphone aspect. Additional subtasks such as sarcasm detection is also implemented. The full pipeline combines various methods such as knowledge-based, transformer-based and rule-based methods. This design is motivated by the inherent complexity of sentiment analysis as a “suitcase research problem”, where a single monolithic classifier is insufficient to handle the full spectrum of linguistic phenomena present in real-world social media comments.

Subjectivity Detection

The first stage of the pipeline performs subjectivity detection to separate comments into ‘Objective’ and ‘Subjective’, before sentiment classification is applied to opinionated content (subjective comments).

A hybrid approach is employed for this task:

- 1) The primary classifier is *GroNLP/mdebertav3-subjectivity-english*, a fine-tuned multilingual DeBERTa-v3 model developed for the CLEF 2023 CheckThat! Lab Task 2 on subjectivity detection in news articles, achieving a macro F1 of 0.77 on the competition dataset [1][2]. Placed third in the competition, this model demonstrates competitive performance relative to the state of art subjectivity detection. The model is implemented via Hugging Face pipeline and it predicts whether a comment is subjective or objective along with a confidence score.
- 2) To improve robustness, a SenticNet-based fallback is used. SenticNet is a commonsense knowledge base that assigns polarity scores to common concepts and words [3]. Its knowledge-driven approach offers broad domain coverage and has demonstrated competitive performance against other lexicals in sentiment analysis research [3]. For low-confidence predictions of the primary classifier (confidence < 0.7), the average absolute polarity magnitude

of words in the comment is computed using SenticNet. If this exceeds a threshold of 0.3, the comment is reclassified as Subjective, otherwise, the original model prediction is retained. Both thresholds were determined empirically through preliminary testing on a sample of the evaluation dataset. Using SenticNet provides an explicit lexical sentiment signal when the transformer model is uncertain.

This combination leverages the complementary strengths of both deep learning which has contextual understanding, and knowledge-based methods which provide explicit sentiment signals, making the subjectivity decision more robust than either approach alone. Comments classified as ‘Objective’ are subsequently excluded from downstream sentiment models and are assigned a default neutral sentiment, ensuring that only opinionated content contributes to sentiment analysis.

Sarcasm Detection

Following subjectivity detection, the pipeline incorporates sarcasm detection to identify and adjust the polarity of subjective comments whose literal wording may not reflect their true sentiment. The classifier used is *surrey-nlp/bertweet-base-finetuned-SARC-DS*, a BERTweet-based model finetuned on the SARC dataset which is a large-scale Reddit corpus of labelled sarcastic and non-sarcastic comments [4]. BERTweet, being pre-trained on social media text, provides a more domain-relevant linguistic foundation than general-purpose transformer models for this task, achieving a validation accuracy of 0.7636 on the SARC evaluation set [4].

The inclusion of this is particularly impactful for our task, due to the nature of YouTube comments, where users may express dissatisfaction through sarcasm, which typically employs positive wording to convey negative intent or vice versa. This mismatch between lexical polarity and true sentiment can cause downstream sentiment models to misinterpret the true polarity of a comment. In such detected cases, sarcastic comments have their final sentiment label reversed.

Aspect Detection

After subjectivity detection, the pipeline determines whether a subjective comment is about smartphones in general or about one specific predefined feature (aspect). We define nine fixed smartphone-related aspects in our pipeline: camera, battery, display, price, software, design, performance, charging and storage. These aspects were predefined rather than extracted dynamically to ensure consistency and comparability across comments, as open-ended aspect extraction can introduce noise. Aspect detection itself is performed through keyword matching based on a set of keywords that define each aspect space. For example, ‘camera’ aspect is defined by the following keywords: camera, photo, photos, video, videos, zoom, selfie, lens, sensor, dynamic range, image quality.

Using keyword matching for aspect extraction is a deliberate design choice as while neural Aspect Term Extraction (ATE) models exist, they may not align well with predefined aspect categories required for this project. In contrast, keyword matching offers better precision and full transparency over our aspect definitions.

If a comment contains exactly one specific aspect, it is routed into the corresponding aspect category. If it contains no specific aspect keywords or contains keywords spanning multiple aspects, it is grouped under an ‘overall’ aspect. Such comments are handled by the general sentiment classification branch, while cleaner single-aspect comments are sent to the aspect-based sentiment analysis (ABSA) branch.

Table 4 shows the aspect distribution of the final sentiment-ready corpus. The largest category is overall with 3,177 comments, followed by camera with 1,462 and battery with 835. Less frequent categories such as charging with 131 and storage with 62 are much smaller. This reveals an important property of the corpus, that although the crawling strategy captures a broad set of aspects, the actual distribution of user discussion is naturally imbalanced.

Table 4: Comment category distribution in comments_relevant.csv

Category	Count
overall	3177
camera	1462
battery	835
display	607
price	585
software	534
design	223
performance	180
charging	131
storage	62

Hybrid Sentiment Classification Approach

General Sentiment Analysis

When no single specific aspect is isolated from subjective comments (comments are classified as ‘overall’), it is reduced to a global sentiment classification task. These comments are processed using a transformer-based sentiment classifier, *cardiffnlp/twitter-roberta-base-sentiment-latest*, via the Hugging Face pipeline [5][6]. The model predicts one of the three sentiment classes — negative, neutral and positive — for each comment and assigns the corresponding confidence score for prediction.

This RoBERTa model was chosen for this branch as it is well-suited for sentence-level (global) sentiment classification, particularly on social media content, making it suitable for our task. As a

transformer model trained with optimized pretraining procedures over BERT, RoBERTa is a strong and widely used baseline for document and sentence-level sentiment tasks, able to capture contextual meaning effectively across variations in language, tone and expression. The model is integrated into the TweetNLP library, which provides state-of-the-art language models specialized for social media NLP tasks including sentiment analysis [7].

Aspect-based Sentiment Analysis (ABSA)

Subjective comments associated with one of the nine predefined smartphone-aspects are processed using ABSA instead of standard sentiment classification. This is implemented using a Microsoft’s DeBERTa-v3-based model (*yangheng/deberta-v3-base-absa-v1.1*), which takes a text-aspect pair as input and predicts sentiment — negative, neutral, positive — specifically with respect to that aspect through the Hugging Face pipeline [8]. This approach enables more fine-grained analysis compared to global sentiment classification, which is particularly important for smartphone review comments that are feature-focused. The model was chosen as it is a state-of-the-art transformer-based ABSA model, built on DeBERTa’s improved attention mechanisms and pretrained on multiple large-scale ABSA datasets, making it well-suited for capturing aspect-specific sentiment [8].

To better align the model with our task’s predefined aspect categories, the DeBERTa-v3-base ABSA model was further fine-tuned on a remapped SemEval 2014 Task 4 Laptop dataset [9]. This dataset was chosen as it contains aspect-level sentiment annotations for consumer electronics product reviews, making it the most domain-relevant publicly available benchmark for fine-tuning our ABSA model. Firstly, as a preprocessing step, rows with ‘conflict’ labels were removed to ensure a consistent three-class sentiment setup (negative, neutral, positive). SemEval aspect terms were then mapped to our nine target aspects using semantic similarity between embeddings, with a minimum similarity threshold of 0.5 applied, to ensure consistency between training data and the classification pipeline’s aspect schema. This yielded about 1517 samples after filtering. The model was subsequently fine-tuned on this adapted dataset with a 85/15 train-validation split using the three-class sentiment setup, with performance optimized using macro F1 and early stopping.

The final pipeline applies this fine-tuned DeBERTa model to all aspect-specific subjective comments, ensuring that sentiment is evaluated in relation to the correct feature in the comment. This combination of pretrained ABSA architecture and task-specific fine-tuning allows the system to produce more accurate and context-aware sentiment predictions at the aspect level.

4.2 Data Preprocessing: Design & Justification

Initially, it was assumed that preprocessing was an essential and non-trivial step in the pipeline due to the highly informal and noisy nature of YouTube comments, where raw text is largely unsuited for direct input into classification models. As such, earlier iterations of the pipeline consisted of multiple ordered stages, including text cleaning, normalization, tokenization, POS tagging, lemmatization, and stopword removal.

However, this design was later revised in the final pipeline to better align with the requirements of transformer-based models. Since the downstream classifiers (DeBERTa, RoBERTa for sentiment)

employ subword tokenization methods such as WordPiece or Byte Pair Encoding (BPE), applying manual tokenization or lemmatization beforehand can disrupt their internal representations and degrade performance. This is applied consistently across all transformer-based classifiers in the pipeline, including those used for sarcasm and subjectivity detection, which similarly rely on internal tokenization. These models are designed to process natural language text directly and excessive preprocessing may remove important morphological and contextual signals. The final preprocessing pipeline therefore is streamlined to focus on a set of targeted, domain-specific normalization steps, each designed to preserve the natural structure of the text while ensuring compatibility with transformer-based models.

Emoji Normalization

Social media comments contain heavy use of emojis as emotional signals, which mostly carries direct sentiment values. Rather than discarding these, they are converted to readable text tokens using the ‘emoji’ library (e.g. an emoji representing a fire = emoji_fire). Discarding emojis would result in a significant loss of sentiment signal, particularly in shorter comments where emojis may be the primary indicator.

Slang & Abbreviation Expansion

Internet text is dense with abbreviations (such as ‘idk’, ‘tbh’, ‘imo’, etc.) that are entirely opaque to models trained on formal text. A curated slang dictionary of 40+ entries is maintained, compiled specifically for smartphone-review discourse and to be expanded before punctuation removal. This is a conscious decision again, since apostrophes in contractions like *can’t* must be preserved during expansion. This step is important for both mDeBERTa (subjectivity detection) and RoBERTa (sentiment analysis), whose tokenizers may otherwise fragment or misrepresent tokens containing informal expressions.

Basic Text Cleanup

Text is normalized by correcting encoding issues (e.g. mojibake) using the ‘ftfy’ library with ‘NFKC’ normalization, and removing whitespace/zero-width characters which are common in copy-pasted web text. URLs and user mentions are replaced with placeholder tokens (“URL”, “@user”) to preserve sentence structure. Curly quotes are also normalized to straight quotes for tokenizer consistency. Notably, normalization of casing was not done as transformers are case-sensitive and benefit from capitalization cues.

Phone Model Normalization

This is a domain-specific feature. Comments often reference the same device in inconsistent formats (e.g. “iPhone 15 Pro Max”, “iphone15promax”, “i phone 15 pro”) which would otherwise be treated as entirely different tokens without normalization. Regex-based rules are used to standardize all references to a canonical form (e.g. “iPhone15Pro”). This is important for downstream classification tasks like aspect detection, where keyword matching must reliably identify device-specific aspects

Overall, the preprocessing strategy is kept deliberately lightweight and domain-specific, focusing on preserving sentence structure and sentiment-related contextual elements while maintaining compatibility with transformer-based models. By avoiding excessive text preprocessing, and instead normalizing only critical elements, the pipeline retains the contextual richness of the original text, allowing our pipeline to fully leverage the power of transformer models.

4.3 Evaluation Dataset

The evaluation dataset was constructed by randomly sampling 1000 comments from the crawled corpus to ensure a representative subset for performance assessment. The distribution of comments across different aspects is presented in Table 5, showing coverage of both overall and aspect-specific categories.

Table 5: Aspect Distribution in the 1000-Row Annotation Candidate Set (annotation_candidates.csv)

Aspect	Count
overall	396
camera	205
battery	116
price	72
software	67
display	52
design	29
performance	25
storage	20
charging	18

Two annotators independently labelled the dataset, assigning ground truth sentiment labels (negative, neutral, positive) for each comment. For comments categorized under overall, annotators evaluated the sentiment of the comment as a whole. For aspect-specific comments, annotators assessed sentiment with respect to the given aspect, ensuring that the assigned label reflected the opinion towards that particular feature rather than the entire comment. The initial inter-annotator agreement was 93.3%. The disagreements in labels between both annotators were resolved by a third annotator, ensuring a consistent and reliable final ground truth dataset.

4.4. Evaluation Metrics

To evaluate the effectiveness of the pipeline, the final classification pipeline was evaluated against the manually annotated dataset with standard multi-class classification metrics: accuracy (or agreement percentage), precision, recall and F1-score, along with macro-averaged and weighted-averaged scores.

The final classification pipeline achieves:

- Accuracy: 54.7%
- Macro F1-score: 0.411
- Weighted F1-score: 0.547

These results show average performance, with the gap between macro and weighted F1 suggesting that the model performs better on more frequent classes while struggling with balanced performance across all classes.

Table 6 presents the per-class performance of the overall classification pipeline.

Table 6: Per-class performance of overall classification pipeline

Class	Precision	Recall	F1-score
Negative	0.608	0.528	0.565
Neutral	0.593	0.490	0.536
Positive	0.471	0.634	0.541

The model demonstrates varying performance across sentiment classes:

- The negative class achieves the highest F1-score and precision, indicating relatively strong detection of explicit complaints or dissatisfaction. This is expected as negative sentiments in product reviews are often expressed more clearly and consistently, making them easier for the sentiment models to identify.
- The neutral class shows the lowest recall, suggesting difficulty in identifying ambiguous or factual comments. This is compounded by the pipeline's subjectivity filter, which assigns objective comments a neutral sentiment label by default, potentially conflating rule-assigned neutrals with genuinely neutral opinions and reducing the coherence of the neutral class overall.
- The positive class achieves the highest recall but lower precision, indicating that the model tends to over-predict positive sentiment. This may be partly attributable to the sarcasm detection component, where false positives cause genuinely negative comments to be flipped to positive, inflating positive predictions and reducing precision.

The relatively lower macro F1-score (0.411) compared to the weight F1-score (0.547) highlights class imbalance and uneven performance across categories.

4.5 Random Accuracy Test

To further assess performance of the pipeline, a random sample of 10 comments from the remaining dataset (outside the evaluation set) was manually inspected as seen in Table 7. The predicted sentiment

labels were compared against human judgement, yielding an approximate accuracy of 50%. This provides a sanity check of the model's performance on unseen and uncured data.

Table 7: Accuracy test of 10 randomly sampled comments

Text	Predicted Label	Ground-truth (human) Label
“Samsung hate the plus users. They try to bully you ibto buying the ultra model. So GalaxyS26 has to somehow replace the plus model. Samsung is a confusing company.	positive	negative
The GOAT strikes again!!! Could you give some guidance on which cases to buy for GalaxyS26?	positive	positive
I'm using a base GalaxyS24 I bought on eBay for \$300 and it's been the most reliable phone I've had in years laughing out loud. S24U USB port broke, Pixel9Pro USB port broke and waterproofing failed, Z Fold 7 outer display failed. I just wish the base models had a bigger battery and better cameras. They've been so stagnant for years.	positive	neutral
@USER i got a 23FE, superior battery life, newer chipset with better thermal management. that 22 gets hot. i found one for 180\$ and use it for gaming	positive	positive
@USER also has a poor battery... why cannot Samsung give a 5000+ MAH battery... its sad	positive	negative
We'in 2026 and there's still no full screen experience (virtual buttons) and samsung wants 900 for this shit	positive	negative
So basically you going to buy same s23U there no major upgrades same battery same camera sensors except privacy display and latest processor	neutral	negative
@USER nope, the exynos usually have better camera quality	neutral	neutral
when we thought this RAMergeddon aren't worse enough... even Samsung BETRAY Samsung themselves	negative	negative
oh my god the GalaxyS10+ was prefect, I still have saved pictures i took with that phone, that are still absolutely amazing	positive	positive

The random accuracy test here provided a qualitative validation of the model's performance on unseen data. Out of 10 randomly sampled comments, it achieved half accuracy. There is a tendency in over-predicting positive sentiment here, especially in texts that contain mixed opinions.

Misclassification was common when distinguishing neutral and negative sentiments, where context-dependent language posed challenges.

4.6 Performance Metrics and Scalability

Since the total runtime of the pipeline is composed of multiple sequential stages, each contributes differently to overall latency. Table 8 provided a detailed breakdown of the time taken for each stage for the pipeline under CPU processing speed and batching (batch size = 16) performed for all transformer inferences.

Table 8: Latency breakdown of each stage and overall classification pipeline

Stage	Records	Elapsed seconds (s)	Records processed per second (s)	Seconds taken per record processed (s)
Subjectivity: GroNLP model	7796	1342.658	5.8064	0.172224
Subjectivity: SenticNet fallback	4331	0.6473	6690.87	0.000149
Sarcasm detection	7796	659.4837	11.8214	0.084593
RoBERTa-base-sentiment	2883	269.6139	10.6931	0.093519
DeBERTa-base-finetuned ABSA	4140	669.9773	6.1793	0.16183
TOTAL (cumulative load across all stages)	26946	2942.38	9.1579	0.109195

This may be summarized as follows:

- Text preprocessing: lightweight and fast, contributing negligible latency per sample (excluded in Table 8)
- Subjectivity detection: moderate latency due to two-step hybrid approach with transformer inference being the dominant contributor, while the SenticNet fallback adds negligible overhead as a lightweight lexical lookup
- Aspect detection (keyword matching): negligible latency (excluded in Table 8)
- General sentiment classification (RoBERTa): moderate latency
- ABSA (Fine-tuned DeBERTa): highest latency due to more complex input (text + aspect)

The pipeline achieves an average processing time of approximately 0.109 seconds per sample. With regards to scalability, it demonstrates reasonableness yet at the same time, becomes constrained by the computational costs of transformer inferences throughout the pipeline. Costs are currently in linear time, as the scaling is clearly in linear relation with regards to the number of input comments.

5. Innovations in the Classification Pipeline

Beyond a baseline sentiment classification approach, as described in earlier sections, several enhancements were introduced to improve robustness and fine-grained understanding of opinions. These include hybrid classification techniques, ABSA and sarcasm detection. These innovations aim to address key limitations of traditional sentiment analysis, particularly in handling noisy, multi-faceted and ambiguous user-generated comments in social media context.

Fine-grained Classification (Aspect-based Sentiment Analysis)

The classification pipeline incorporates ABSA for comments tied to specific smartphone features.

Standard sentiment models collapse all opinions into a single label, losing important detail. In contrast, ABSA enables the system to capture sentiment at the aspect level, allowing the search engine to present more relevant and tailored results based on user interests and queries of specific aspects of smartphones.

Example:

“I wish they upgraded the camera instead of putting those microscopic sensors in the 5th time.”
(Comment ID = “UgzRjTFhPYTJzCkcs054AaABAg”)

The camera aspect is detected from the comment using keyword detection and the ABSA model evaluates the sentiment specifically towards that aspect rather than the comment as a whole, correctly predicting negative sentiment with a confidence of 0.5582.

Hybrid Classification (Symbolic and Subsymbolic AI in Subjectivity Detection)

For subjectivity detection task, the pipeline adopts a hybrid approach by combining a transformer-based model (mDeBERTa) with a SenticNet knowledge base fallback.

Pure neural models may struggle with low-confidence or ambiguous inputs, while knowledge-based methods lack contextual understanding. Additionally, the GroNLP mDeBERTa subjectivity model used in the classification pipeline is trained primarily on newspaper-style sentences, which differ from the informal, noisy nature of YouTube comments in our dataset. As such, incorporating SenticNet knowledge base as a fallback helps balance this domain mismatch by providing explicit lexical sentiment signals, improving robustness of classification.

Example:

“my GalaxyS21 still works fine. only issue is I had to get the screen and camera replaced so I think it lost some water resistance..” (Comment ID = “Ugz4_x5u1gfEr3ZMIVB4AaABAg”)

The GroNLP model incorrectly classifies this as objective, but the SenticNet fallback correctly overrides this to subjective, demonstrating how the hybrid approach compensates for the model's limitations.

Enhanced Classification (Sarcasm Detection)

YouTube comments have the tendency to contain sarcastic expressions, where positive words are used in a negative context. In such cases, standard sentiment classification models may incorrectly assign a positive label due to the presence of positive keywords, despite the overall intent being negative.

To address this, the sarcasm detection component is implemented to identify such patterns and flip the polarity label accordingly to improve the accuracy of sentiment interpretation in our data.

Example:

“So all you need is a \$5 privacy screen protector to upgrade the GalaxyS25 to a GalaxyS26? Got it! emoji_thumbs_up_light_skin_tone” (Comment ID = “UgyfURldvoR2gfv3efl4AaABAg”)

This comment is initially classified by the ABSA model as positive, possibly due to the presence of positive phrases such as “Got it!” and a positive sentiment emoji. The sarcasm detection model correctly identifies this as a sarcastic remark, leading to the inversion of the sentiment label from positive to negative.

Ablation Studies

To quantify the contribution of each proposed innovation, an ablation study was conducted by incrementally adding components to the pipeline and evaluating performance at each stage as seen in Table 9. The five runs were designed to isolate the contribution of aspect-based sentiment analysis (ABSA), subjectivity filtering and sarcasm detection to overall classification performance.

Table 9: Ablation runs and results

Run	Pipeline	Accuracy	F1 neg	F1 neu	F1 pos	Macro F1
1	RoBERTa-base-sentiment on all comments	0.595	0.633	0.5533	0.5968	0.4458
2	+ ABSA (RoBERTa for overall, DeBERTa-base-finetuned for aspect)	0.589	0.6496	0.5235	0.5822	0.4388
3	+ Subjectivity filtering (GroNLP model only)	0.548	0.5888	0.547	0.5095	0.4113
4	+ Subjectivity filtering (GroNLP + SenticNet hybrid)	0.577	0.6287	0.5362	0.5637	0.4321
5	+ Sarcasm detection (full pipeline)	0.547	0.5654	0.5362	0.5407	0.4106

Effect of ABSA routing (Run 1 → Run 2):

Introducing aspect-aware routing in Run 2 yields a marginal decrease in accuracy from 0.595 to 0.589. Rather than applying RoBERTa uniformly to all preprocessed comments, Run 2 routes smartphone aspect-specific comments to the fine-tuned DeBERTa-base ABSA model while retaining RoBERTa for

general overall comments. Notably, F1 for the negative class improves, suggesting that the ABSA model better captures negative sentiment in aspect-specific comments. However, this is offset by drops in both neutral and positive F1 indicating that the ABSA model struggles more with these classes. The drop in overall accuracy can be partly attributed to domain mismatch. Despite the fine-tuning on mapped SemEval dataset, the underlying DeBERTa-base model was pre-trained on data that differs in style, length and linguistic features from YouTube comments. This may limit the model's ability to generalize to our corpus, particularly for neutral and positive sentiment.

Effect of hybrid subjectivity filtering (Run 3 → Run 4):

Run 4 replaces the GroNLP model-only subjectivity filtering with a hybrid approach that applies SenticNet as a fallback for low-confidence GroNLP predictions, recovering 0.029 in accuracy from 0.548 to 0.577. The GroNLP model's weakness on informal YouTube comments leads to aggressive over-filtering in Run 3, with systematic errors concentrated in the negative and positive classes (decrease in negative and positive F1) as comments are incorrectly forced to neutral, while neutral F1 is inflated by the increased volume of forced neutral predictions. SenticNet's lexical polarity signal is complementary in this regard, directly measuring sentiment presence at the token level without relying on learned subjectivity judgements. The recovery in negative and positive F1 in Run 4 confirms that the hybrid approach corrects genuine over-filtering errors rather than introducing noise. It is nonetheless worth noting that subjectivity filtering still produces a net accuracy reduction relative to Run 2, as neutral F1 remains below Run 2 levels. This highlights a probable and inherent limitation of hard neutral mapping for objective comments.

Effect of sarcasm detection (Run 4 → Run 5):

Adding sarcasm detection in Run 5 applies a polarity flip to comments classified as sarcastic, reversing their predicted sentiment label, and produces a 0.030 drop in accuracy. This shows that the effectiveness of this task is highly dependent on the domain alignment between the model's training data and the target corpus. The sarcasm model used was trained on Twitter and Reddit data, where sarcasm patterns may vary as compared to YouTube comments about smartphone descriptions. This domain mismatch could have resulted in the high false positive rate, where correct predictions are flipped to incorrect more often than incorrect predictions are corrected. This is reflected in F1 drops across all three sentiment classes in Run 5 relative to Run 4, with the negative class showing the sharpest decline followed by positive, while neutral F1 remains unchanged. The unchanged neutral F1 is expected, as sarcasm flips only affect non-neutral predictions.

Conclusion of classification pipeline

Overall, the pipeline's moderate accuracy reflects the inherent difficulty of sentiment classification on domain-specific text, where domain mismatch across the constituent models, the hard neutral mapping applied to objective comments and sarcasm detection false positives collectively constrain performance. Nonetheless, the pipeline provides a structured, interpretable approach to aspect-aware sentiment analysis that goes beyond simple polarity classification. The ablation study demonstrates that the ABSA routing and hybrid subjectivity filtering each contribute positively in targeted ways, while

sarcasm detection introduces a net accuracy reduction due to what we believe was a domain mismatch. This is a limitation that we acknowledged as a direction for future work and improvement.

Links for Submissions

Youtube Video

The link to the YouTube video that serves as both a quick explanation and demo of the search engine:

https://youtu.be/_MGWTAPU5uA

Dataset

Link to only the data we used for the search engine project:

<https://drive.google.com/file/d/1L1Za8QC0ViNuJE0WB2tBVKE5fxJnvxWV/view?usp=sharing>

Source Codes and Libraries

Link to the entire repository (including the data and model checkpoints) to replicate run on local machine:

<https://drive.google.com/file/d/1tWQwoyCerB2rIkJsCw-pLpALj5cQq5ON/view?usp=sharing>

Appendix A: Screenshots of UI

Figure 7: UI summary page



Figure 8: UI ranked results page

Index Controls

Rebuild Index

Facets

Reset Filters

Brand

All

Phone Model

All

Category

All

Enable Date Range

Deploy

SC4021 Opinion Search Engine

Search

Query

iphone 17 pro

Run Query

Model detected -> bucket fixed to iPhone 17 Pro

Brand detected -> family fixed to Apple

Result Count

539

Latency (ms)

187.78

Query

iPhone 17 Pro

Summary

Ranked Results

Compare Sentiment

About 539 results

Prev

1

2

3

4

5

6

7

...

54

Next

Page 1 of 54

#1 iPhone 17 Pro Review - Something's Missing

comment

positive

iPhone 17 Pro - Marques Brownlee - 2025-09-22 - category: overall - weighted 0.726 - bm25 33.527 - 5

Going from the iPhone 13 Pro to the iPhone 17 Pro, I feel like I have a completely new iPhone. Not used to the Dynamic Island and all the new features but I love the new camera quality.

query seed: iphone 17 pro review

#2 iPhone 17 Pro Review - Something's Missing

reply

neutral

iPhone 17 Pro - Marques Brownlee - 2025-09-23 - category: battery - weighted 0.361 - bm25 33.301 - 0

I'm upgrading from 13 pro max to 17 pro...how is the battery??

query seed: iphone 17 pro review

#3 iPhone 17 Pro Review: Paradox in a Box!

comment

positive

iPhone 17 Pro - Marques Brownlee - 2025-09-27 - category: overall - weighted 0.735 - bm25 33.961 - 59

The greatest paradox of the iPhone 17 Pro: the most "pro" iPhone ever... is also the most mainstream phone Apple has ever made.

query seed: iphone 17 pro review

#4 iPhone 17 Pro Review: Paradox in a Box!

comment

positive

iPhone 17 Pro - Marques Brownlee - 2025-09-27 - category: overall - weighted 0.741 - bm25 33.912 - 10

Wild how the iPhone 17 Pro manages to be the most "Pro" iPhone ever while still being the most mainstream. Love the thicker design with bigger battery + better cooling

query seed: iphone 17 pro review

#5 iPhone 17 Pro Review: Paradox in a Box!

comment

positive

iPhone 17 Pro - Marques Brownlee - 2025-09-30 - category: overall - weighted 0.727 - bm25 34.082 - 1

This intro is better than apple's iPhone 17 pro intro

query seed: iphone 17 pro review

#6 iPhone 17 Pro Review - Something's Missing

comment

neutral

iPhone 17 Pro - Marques Brownlee - 2025-09-23 - category: camera - weighted 0.723 - bm25 33.528 - 0

Has anyone actually compared if the iPhone 17 pro (with its antenna surrounding the camera bump) is able to work better than a standard iPhone 17 in a location with marginal cellular signal reception?

query seed: iphone 17 pro review

#7 iPhone 17 vs Samsung S25 vs iPhone 17 Pro - Battery Test

comment

positive

iPhone 17 Pro - PhoneStar - 2025-09-20 - category: overall - weighted 0.684 - bm25 31.704 - 3

surprised and really happy when iphone 17 pro improved battery a lot compare last video (even same ios 26.3.1 version)

query seed: iphone 17 pro battery

#8 iPhone 17 Pro Review: Paradox in a Box!

comment

neutral

iPhone 17 Pro - Marques Brownlee - 2025-02-03 - category: camera - weighted 0.681 - bm25 34.305 - 0

I enjoyed the video on my iPhone 17 Pro Max.

query seed: iphone 17 pro review

#9 iPhone 17 Pro Review - Something's Missing

comment

neutral

iPhone 17 Pro - Marques Brownlee - 2025-09-23 - category: overall - weighted 0.677 - bm25 33.572 - 0

you should make a iPhone 17 pro max vs s25 ultra vs pixel 10 pro xl

query seed: iphone 17 pro review

#10 iPhone 17 Pro Review: Paradox in a Box!

comment

positive

iPhone 17 Pro - Marques Brownlee - 2025-09-28 - category: overall - weighted 0.676 - bm25 34.689 - 5

Soooo underrated. Coming from the 16 pro... I got the black iPhone 17 and I looveeee it Highly recommend the iPhone 17

query seed: iphone 17 pro review

Prev

1

2

3

4

5

6

7

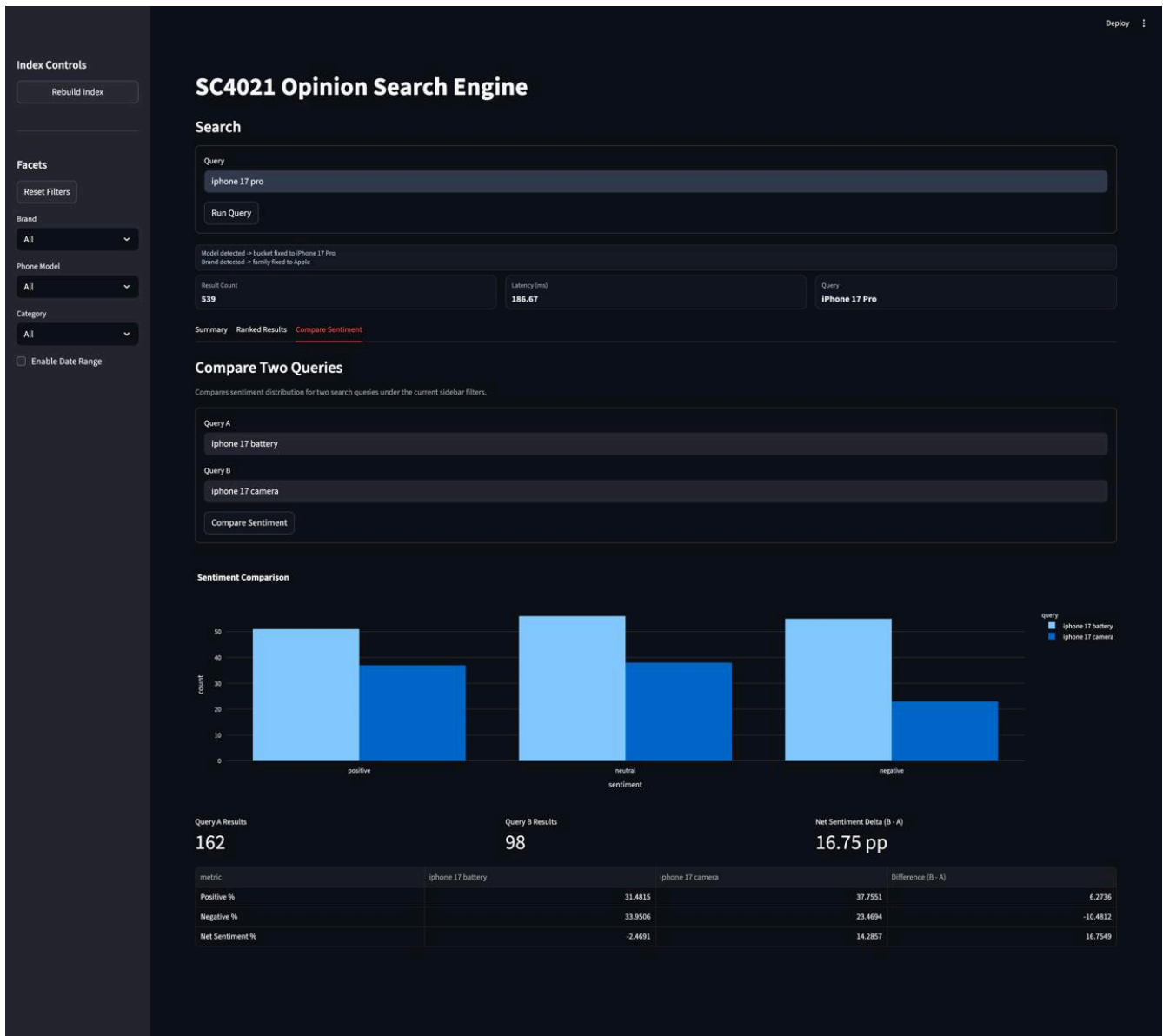
...

54

Next

Page 1 of 54

Figure 9: UI compare sentiment page



References

- [1] “GroNLP/mdebertav3-subjectivity-english · Hugging Face,” *Huggingface.co*, 2023.
<https://huggingface.co/GroNLP/mdebertav3-subjectivity-english>
- [2] F. Ruggeri, F. Antici, A. Galassi, K. Korre, A. Muti, and A. Barrón-Cedeño, “On the Definition of Prescriptive Annotation Guidelines for Language-Agnostic Subjectivity Detection.” [Online]. Available: <https://ceur-ws.org/Vol-3370/paper10.pdf>
- [3] E. Cambria, Q. Liu, S. Decherchi, F. Xing, and K. Kwok, “SenticNet 7: A Commonsense-based Neurosymbolic AI Framework for Explainable Sentiment Analysis,” *ACLWeb*, Jun. 01, 2022.
<https://aclanthology.org/2022.lrec-1.408/>
- [4] “surrey-nlp/bertweet-base-finetuned-SARC-DS · Hugging Face,” *Huggingface.co*, 2022.
<https://huggingface.co/surrey-nlp/bertweet-base-finetuned-SARC-DS>
- [5] “cardiffnlp/twitter-roberta-base-sentiment-latest · Hugging Face,” *huggingface.co*.
<https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest>
- [6] D. Loureiro, F. Barbieri, L. Neves, L. E. Anke, and J. Camacho-Collados, “TimeLMs: Diachronic Language Models from Twitter,” *arXiv:2202.03829 [cs]*, Apr. 2022, Available: <https://arxiv.org/abs/2202.03829>
- [7] “TweetNLP,” *GitHub*, Apr. 15, 2023. <https://github.com/cardiffnlp/tweetnlp>
- [8] “yangheng/deberta-v3-base-absa-v1.1 · Hugging Face,” *huggingface.co*.
<https://huggingface.co/yangheng/deberta-v3-base-absa-v1.1>
- [9] M. Pontiki, D. Galanis, J. Pavlopoulos, H. Papageorgiou, I. Androutsopoulos, and S. Manandhar, “SemEval-2014 Task 4: Aspect Based Sentiment Analysis,” *ACLWeb*, Aug. 01, 2014.
<https://www.aclweb.org/anthology/S14-2004/>